

$$A+B=C$$

Unsupervised Learning

K-Means

$$\sqrt{a}$$

$$2^{-}$$





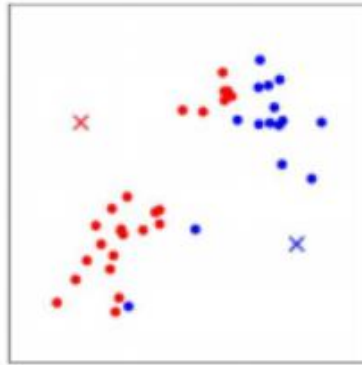
K-Means



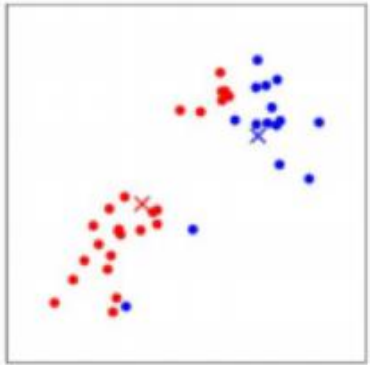
(a)



(b)



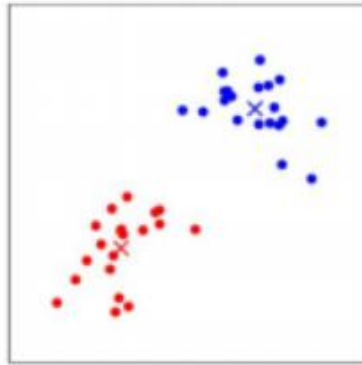
(c)



(d)



(e)



(f)

- It tries to minimize the variance within clusters and maximize the variance between clusters.





K-Means

1. Initialize cluster centroids

Pick k initial centroids $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ (randomly or by some heuristic).

2. Repeat until convergence (no change in assignments or centroids):

(a) Cluster assignment step:

For each data point $x^{(i)}$, assign it to the nearest centroid:

$$c^{(i)} := \arg \min_{j \in \{1, \dots, k\}} \|x^{(i)} - \mu_j\|^2$$

(b) Centroid update step:

For each cluster j , update its centroid as the **mean of all points assigned to it**:

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}$$

where $1\{c^{(i)} = j\}$ is the indicator function(1 if point i is in cluster j , else 0).

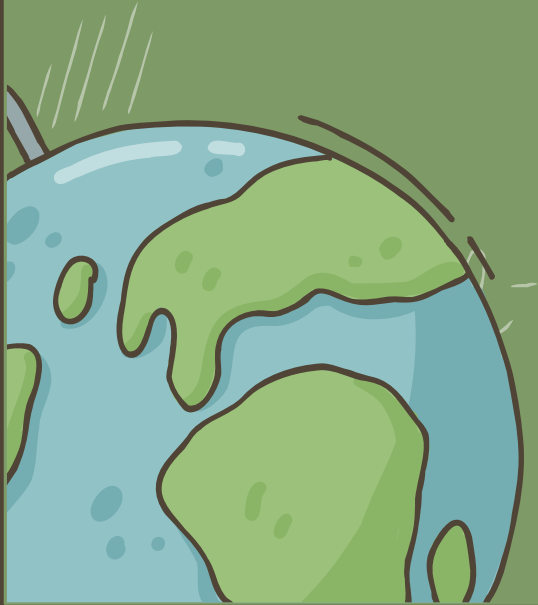


$$A+B=C$$

Statistical Learning

—

$$\sqrt{a}$$





Bayesian learning

Consider a simple example. Our favorite surprise candy comes in two flavors: cherry (yum) and lime (ugh).

h_1 : 100% cherry,

h_2 : 75% cherry + 25% lime,

h_3 : 50% cherry + 50% lime,

h_4 : 25% cherry + 75% lime,

h_5 : 100% lime

Bayesian learning simply calculates the probability of each hypothesis, given the data, Bayesian learning and makes predictions on that basis. T





Bayesian learning

- In Bayesian learning, we compute the posterior probability of a hypothesis given the data:

$$P(h_i | d) \propto P(d | h_i) P(h_i)$$

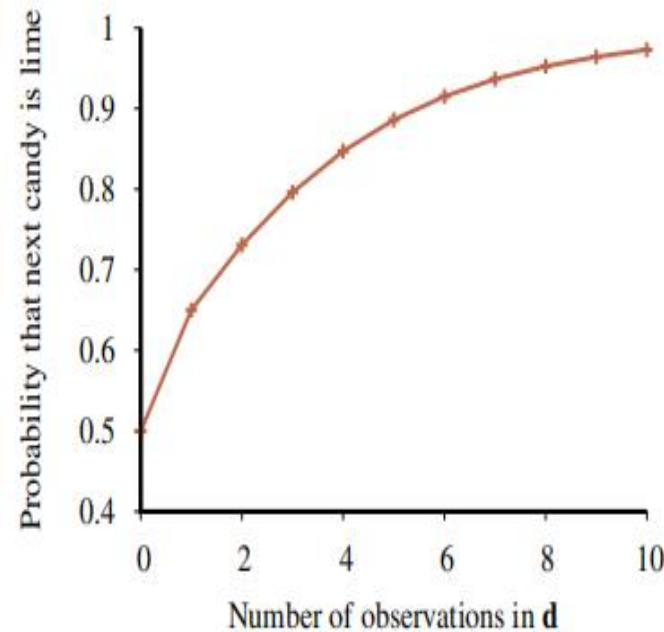
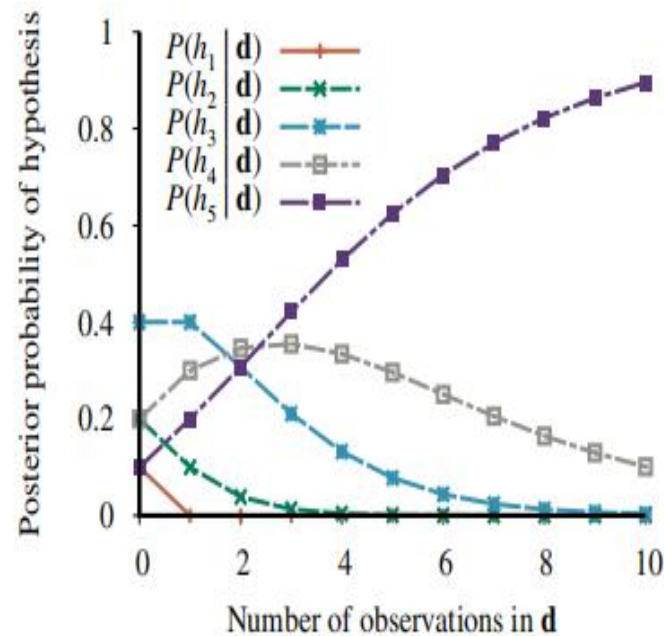
$P(d | h_i)$ = Likelihood $\rightarrow$ how well hypothesis explains the data.

$P(h_i)$ = Prior $\rightarrow$ how much we believed in this hypothesis before seeing data.

$P(h_i | d)$ = Posterior $\rightarrow$ updated belief after data.

$$\mathbf{P}(X | \mathbf{d}) = \sum_i \mathbf{P}(X | h_i) P(h_i | \mathbf{d}),$$





h_1 : 100% cherry,
 h_2 : 75% cherry + 25% lime,
 h_3 : 50% cherry + 50% lime,
 h_4 : 25% cherry + 75% lime,
 h_5 : 100% lime.

Bayesian prediction eventually agrees with the true hypothesis.



MAP definition

- The Maximum A Posteriori (MAP) estimate chooses the hypothesis with the highest posterior probability:

$$h_{MAP} = \arg \max_{h_i} P(h_i \mid d)$$

$$h_{MAP} = \arg \max_{h_i} P(d \mid h_i) P(h_i)$$

- So unlike Maximum Likelihood (ML), which only maximizes $P(d \mid h_i)$ (ignores the prior), MAP also considers how plausible the hypothesis was before the data.





Maximum Likelihood

- Uniform prior assumption: If we assume all hypotheses are equally likely at the start (i.e., priors are uniform), then: $P(h_i) = \text{constant}$ for all h_i
- So in the MAP equation:
- The prior term $P(h_i)$ cancels out, leaving:
- This is the Maximum Likelihood (ML) estimate

$$h_{MAP} = \arg \max_{h_i} P(d | h_i) P(h_i)$$

$$h_{MAP} = \arg \max_{h_i} P(d | h_i)$$





When does ML work well?

- With large datasets:

The evidence from the data is so strong that it overwhelms priors anyway. So $ML \approx MAP \approx \text{full Bayesian}$.

- With small datasets:

Priors matter a lot, because limited data may mislead. Here, ML can perform poorly (e.g., predicting coin bias from only 1–2 tosses).





Intuition with a toy story

Let's reuse the toy box example

- Priors: cars are more common than balls.
- Data: you hear a rolling sound.
- Likelihoods: both cars and balls fit rolling, but cars fit slightly better.

ML: chooses the toy that best matches the rolling sound (maybe the ball if it rolls most clearly).

MAP: chooses the toy that matches the rolling sound and was already more likely to be in the box (probably car).





Fully observable vs hidden variables

- In a fully observable case, all the information we need is available in the dataset.

Example: If we're modeling students' test scores, we might have study hours, sleep hours, and exam marks — all recorded.

- But in many real-world problems, some important causes are not directly observed. These are called latent (hidden) variables.





Disease diagnosis

Imagine medical records:

- What we see (observed): symptoms (fever, cough), doctor's diagnosis, treatment, outcome.
- What we don't see directly (hidden): the true disease causing the symptoms.
- Important: The disease itself is hidden — you never directly “see” it in the data. The diagnosis is only a guess, based on symptoms.





Connecting to hidden variables: ML and MAP

Disease in the medical model

- Example: In the heart disease model, after symptoms are observed:
- ML would say: “Pick the disease that best explains the symptoms.”
- MAP would say: “Pick the disease that best explains the symptoms AND is also more common in the population.”
- So MAP balances data evidence (likelihood) with prior belief (prevalence).



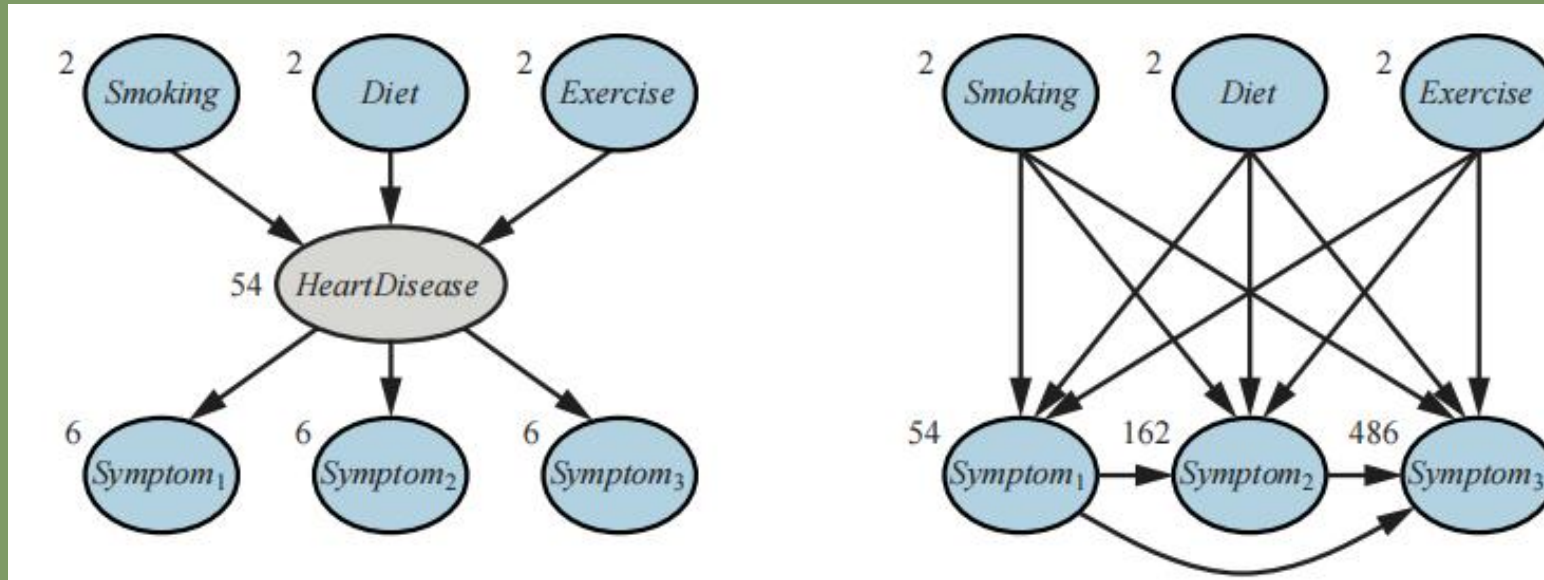


Why hidden variables are useful

- If you try to model everything only with observed variables, the network becomes very messy.
- Without the hidden disease variable, you have to directly connect every symptom to every predisposing factor, which needs hundreds of parameters.
- With a hidden variable (disease), you just connect factors $\rightarrow$ disease $\rightarrow$ symptoms, and suddenly the parameter count is much smaller.



Why hidden variables are useful





Expectation–Maximization (EM)

- “fill in the blanks” for hidden variables while learning.

Intuition:

- E-step (Expectation): Guess the missing (hidden) variables based on what we currently know.
- M-step (Maximization): Update the model parameters as if those guesses were real.
- Repeat until things stabilize.





Hidden Markov Model (HMM)

In an HMM, we have:

- **Hidden states** (latent variables): e.g., the actual weather each day (sunny , rainy , cloudy).
- **Observed data**: what we see, e.g., whether a person carries an umbrella or not.
- The true state (weather) is not observed directly — it's a latent variable. So, HMMs are the perfect example of latent-variable models.





Learning in HMMs

When we train an HMM from data, we need to estimate two sets of parameters:

1. Transition probabilities: $P(\text{State}_t \mid \text{State}_{t-1})$
Emission probabilities: $P(\text{Observation}_t \mid \text{State}_t)$.
(e.g., if it's rainy, $\text{prob}(\text{umbrella}=\text{yes}) = 0.9$).

But... the states are hidden, so we don't directly know which state generated which observation.



$$Q = q_1 q_2 \dots q_N$$

a set of N **states**

$$A = a_{11} \dots a_{ij} \dots a_{NN}$$

a **transition probability matrix** A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$

$$B = b_i(o_t)$$

a sequence of **observation likelihoods**, also called **emission probabilities**, each expressing the probability of an observation o_t being generated from a state q_i

$$\pi = \pi_1, \pi_2, \dots, \pi_N$$

an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$



Learning in HMMs

One person either feels Happy (H) or Sad (S) (hidden states).
We don't directly observe mood, but we observe whether they Sing (Si) or Cry (C) (observations).

Hidden States:

H = Happy

S = Sad

Observations:

Si = Sing

C = Cry

Initial Probabilities:

$P(H)=0.5$, $P(S)=0.5$





Transition and Emission Probabilities

- Transition Probabilities:

$$P(H \rightarrow H) = 0.7, P(H \rightarrow S) = 0.3$$

$$P(S \rightarrow H) = 0.4, P(S \rightarrow S) = 0.6$$

- Emission Probabilities:

$$P(\text{Si} | H) = 0.6, P(\text{C} | H) = 0.4$$

$$P(\text{Si} | S) = 0.1, P(\text{C} | S) = 0.9$$





Observation Sequence

- $O = [Si, C, Si]$





Step 1: Initialization

- $\delta_1(H) = P(H) \times P(Si|H) = 0.5 \times 0.6 = 0.3$
- $\delta_1(S) = P(S) \times P(Si|S) = 0.5 \times 0.1 = 0.05$





Step 2: Recursion

- $t=2$, $O_2=C$:
- $\delta_2(H) = 0.084$
- $\delta_2(S) = 0.081$

- $t=3$, $O_3=Si$:
- $\delta_3(H) = 0.03528$
- $\delta_3(S) = 0.00486$





Step 3: Termination & Backtracking

- Final probability = $\max(0.03528, 0.00486) = 0.03528$
- Best final state = Happy (H)
- Backtracking:
- [H, H, H]





Baum–Welch Algorithm

- Hidden states are not directly observed
- We only observe sequences (e.g., Sing, Cry)
- Goal: Estimate HMM parameters
 - Transition probabilities (A)
 - Emission probabilities (B)
 - Initial distribution (π)





Intuition

- We can't directly count transitions/emissions
- Use Forward–Backward algorithm to estimate expected counts
- Then update parameters iteratively
- Repeat until likelihood converges





E-step: Forward–Backward

- **Forward probability $\alpha_t(i)$:**

Probability of partial observation up to time t and ending in state i .

- **Backward probability $\beta_t(i)$:**

Probability of partial observation from time $t+1$ to end, given state i at time t .





E-step: Forward–Backward

- $\gamma_t(i)$ = Probability of being in state i at time t , given the whole observation sequence.

$$\gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)}$$

- $\xi_t(i,j)$ = Probability of being in state i at time t , and state j at time $t+1$, given the observations.

$$\xi_t(i,j) = \frac{\alpha_t(i)a_{ij}b_j(O_{t+1})\beta_{t+1}(j)}{\sum_i \sum_j \alpha_t(i)a_{ij}b_j(O_{t+1})\beta_{t+1}(j)}$$





M-step: Re-estimation

- Initial distribution:

$$\pi_i = \gamma_1(i)$$

- Transition probabilities:

$$a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

- Emission probabilities:

$$b_j(k) = \frac{\sum_{t=1}^T \gamma_t(j) \mathbf{1}[O_t = k]}{\sum_{t=1}^T \gamma_t(j)}$$





Iteration Process

1. Initialize parameters randomly or uniformly
2. Run Forward–Backward (E-step)
3. Update parameters (M-step)
4. Repeat until convergence
 - Likelihood stops improving significantly

